Learning Minimal-Thrust Corrections Towards the Figure-8 Orbit

Ignacio Blancas Rodriguez
Brown University
ignacio_blancas_rodriguez@brown.edu

Ryan L Yang
Brown University
ryan_l_yang@brown.edu

Patrick Verma
Brown University
ankit_verma@brown.edu

***Abstract*—We consider the problem of guiding three celestial bodies into a stable figure-8 orbit from some initial unstable positions through discrete-time pushes. To do so, we consider two different approaches. First, a three-agent Reinforcement Learning setup trained through PPO with a Centralized critic and Centralized reward but Decentralized inference. Our second approach runs MPC/iLQR in the receding horizon mode to generate expert rollouts to then train a TD3 agent that controls the thrusts of the three celestial bodies. We find that fine-tuning our TD3 agent after training it on expert rollouts yields stronger convergence over shorter training times than our MARL PPO setup.**

GitHub Repository: https://github.com/iblancas/CSCI1470FinalProjectIgnacioPatrickRyan

I. Introduction

A. Motivation

The N-body problem considers the time evolution of N celestial bodies and their gravitational interactions. The task of predicting the positions and velocities of the bodies at any future time-step t has a closed-form solution for $N \leq 2$; however, systems with $N \geq 3$ are generally chaotic. There is ample literature on the problem of predicting the optimal simulation time-step for systems of $N \geq 3$ bodies, with approaches learning energy-error minimizing time-steps for the Hermite integrator via Deep-Q networks [1]. We consider the problem of guiding three planetary bodies into a stable orbit, such as the figure-8 orbit, through discrete-time pushes. At every time-step ΔT an acceleration,

$$a_i = (x_i, y_i)$$

is applied to each of the three bodies. While a bound for this acceleration is not directly enforced, exceeding a chosen bound is penalized in the rewards function.

B. Approaches

We explore two different approaches in the modeling of our system, both of which implement the Hermite integrator for the simulation, through the AMUSE backend in Python. The first approach integrates three independent RL agents with a centralized critic and a centralized GNN generating the embeddings for the agents. We implement a PPO-based training loop and consider two different initializations, one where the bodies are initialized in some fixed chosen positions at the beginning of training, and one where the bodies are initialized in random positions every time a new simulation is started. The second approach is a teacher-guided continuous-control pipeline using MPC/iLQR and TD3. For this second approach, we reduce the problem to a single actor controlling all three celestial bodies to simplify the model.

C. Goals and Results

While the first approach simulates a more natural exploration-based approach towards learning the correct pushes, the solution space of the problem is too large and sparse. For random initial positions, the first approach was able to yield fairly weak convergence, which was mostly just able to increase survival time, but was not able to approximate the Figure-8 orbit. By fixing the initial positions to some known ones, where the system did not become immediately chaotic, but also did not immediately default to a stable orbit, we were able to improve the convergence towards the orbit. However, these were short-term improvements, and after a bounded number of simulation steps, the system descends into chaos.

On the other hand, the second approach greatly reduces the solution search space for the TD3 RL agent, allowing for stronger convergence results in shorter training times. This, however, comes at the cost of solution discoverability through RL, as this approach requires the generation of expert trajectories.

II. Literature Review

As it comes to the N-body problem, most of the existing literature focuses on the problem of adaptive time-stepping for the Hermite integrator. On their paper, Dr. Saz Ulibarrena and Professor. Portegies Zwart [1] propose the use of Reinforcement Learning to predict dynimically produce the time-step parameter for the Hermite integrator in a system of 3 bodies. Their approach tries to resolve three main issues with traditional approaches,

- **Expert knowledge**: Traditional approaches require expert knowledge of astrophysics, and details from the implementation of numerical approximations to adapt the time-step in an informed manner.
- **Fixed hidden parameters**: Some of the parameters used to determine the time-step in traditional approaches stay

fixed throughout the simulation. This fails to capture the rapidly changing dynamics of these chaotic systems.
- **Accuracy-Effort balancing**: Unlike most traditional approaches, their approach dynamically balances simulation accuracy and computation effort, and it does so throughout the simulation, allowing for better tailored results.

To achieve this, they propose the use of a Deep-Q Network to learn the appropriate time-step Δt for the Hermite Integrator for a 3-body setup. The DQN is fed the entire state X of the system, which concatenates the positions and velocities of the three bodies.

The rest of the literature in our citations pertains to the second of our approaches towards the problem of pushing 3 bodies into a stable figure-8 orbit. We utilize the results in [2] and [3] for stage 1 of our pipeline in the second approach, where we apply the iLQR to solve the local finite-horizon control problem, and the standard MPC receding-horizon control ideas in [4]. Finally, we fine-tune our TD3 agent via the techniques proposed in [5].

III. Methodology

A. The Simulator

The system is composed of three bodies in 2D Euclidean Space, each with an associated position and velocity vector, $x_i, v_i \in \mathbb{R}^2$. In the time-dependent setting, we can denote these positions and velocities as $x_t^{(i)}, v_t^{(i)}$, then the goal of the simulator is to produce $p_{t+\Delta t}^{(i)}, v_{t+\Delta t}^{(i)}$. We apply the Hermite integrator, a popular choice for approximating this problem. This is a 4th-order integrator that keeps track of position, velocity, acceleration, and jerk for the system in order to approximate its new positions and velocities. It does so according to the following update rule,

$$x_{\text{pred}}^{(i)} = x_t^{(i)} + v_t^{(i)}\Delta t + \frac{1}{2}a_t^{(i)}(\Delta t)^2 + \frac{1}{6}j_t^{(i)}(\Delta t)^3$$
$$v_{\text{pred}}^{(i)} = v_t^{(i)} + a_t^{(i)}\Delta t + \frac{1}{2}j_t^{(i)}(\Delta t)^2$$

where $a_t^{(i)}$ and $j_t^{(i)}$ are the corresponding acceleration and jerk vectors at time t. Then we compute predicted acceleration and jerk vectors,

$$a_{\text{pred}}^{(i)} = G\sum_{j=1,j\neq i}^{3}\frac{m_j}{(x_{\text{pred}}^{(ij)})^3}x_{\text{pred}}^{(ij)}$$
$$j_{\text{pred}}^{(i)} = G\sum_{j=1,j\neq i}^{3} m_j\left[\frac{v_{\text{pred}}^{(ij)}}{(x_{\text{pred}}^{(ij)})^3} - \frac{3(v_{\text{pred}}^{(ij)}\cdot x_{\text{pred}}^{(ij)})x_{\text{pred}}^{(ij)}}{(x_{\text{pred}}^{(ij)})^5}\right]$$

where m_j is the mass of the j-th body. Finally, we compute the new positions and velocities as follows,

$$x_{t+\Delta t}^{(i)} = x_t^{(i)} + \frac{1}{2}(v_t^{(i)} + v_{\text{pred}}^{(i)})\Delta t + \frac{1}{12}(a_t^{(i)} - a_{\text{pred}}^{(i)})(\Delta t)^2$$
$$v_{t+\Delta t}^{(i)} = v_t^{(i)} + \frac{1}{2}(a_t^{(i)} + a_{\text{pred}}^{(i)})\Delta t + \frac{1}{12}(j_t^{(i)} - j_{\text{pred}}^{(i)})(\Delta t)^2$$

One can trivially add to this the accelerations applied to the bodies by the agents' policies. To ease the implementation, we use the implementation of the Hermite integrator and full simulation provided on the AMUSE package.

B. The Target Orbit

Although the approach could, in theory, be replicated for other stable orbits, we present our findings for the figure-8 orbit. In order to measure convergence to the orbit, we need to encode the different positions in the orbit, velocities (to encode continuity within the orbit), and the phase difference between the bodies (to achieve the appropriate synchronization). The way we do that is by taking a sequence of reference snapshots for one body traversing the orbit choreography, with each snapshot being a vector $s_t \in \mathbb{R}^4$ of the form,

$$s_t = [x(t), y(t), v_x(t), v_y(t)]$$

Then, it is assumed that all three bodies travel at a phase difference of $\frac{2\pi}{3}$ from each other; therefore, their paths are essentially equivalent.

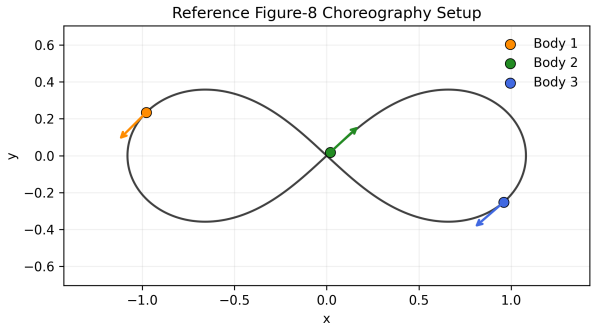

Fig. 1: Reference Figure-8 choreography. The stored target orbit provides position and velocity targets for matching the simulated state at each step.

C. Matching the Orbit

Given the global state of our system, that is, the positions and velocities of the three bodies, we must tackle the problem of assessing "how close" we are to the target figure-8 orbit. At each step, the environment tries every assignment between simulated bodies and reference bodies, then chooses the best match.

The match uses four criteria:

- **Position:** bodies should be close to the reference locations.
- **Velocity direction:** bodies should move in the same direction as the reference choreography.
- **Phase continuity:** the matched point on the Figure-8 should move smoothly forward over time.
- **Assignment continuity:** the same simulated body should keep matching the same reference track unless switching gives a much better fit.

The assignment-continuity penalty prevents the matcher from rapidly swapping body identities between frames. This makes the reward reflect a cleaner physical choreography, where each body settles into a consistent role in the Figure-8 motion.

Through this criteria, the matcher is able to compute some optimal ordering of the bodies and some $k_t \in [0, N)\cap\mathbb{Z}$, where

N is the number of snapshots taken from the reference orbit. It then uses this ordering and k_t to compute the following observation vector, $o_t \in \mathbb{R}^{17}$, which will be used to compute the penalty,

$$\begin{aligned} o_t = &[x_1(t), y_1(t), x_2(t), y_2(t), x_3(t), y_3(t), \\ &m_1, m_2, m_3, \\ &\sin\frac{2\pi k_t}{N}, \cos\frac{2\pi k_t}{N}\Big] \end{aligned}$$

D. Model Architecture

As it came to our first approach, we opted for a Multi-Agent Reinforcement Learning setup with a Centralized Critic for training and Decentralized inference. Each of the three bodies will be controlled by some agent, which will determine its acceleration via some policy $\pi^{(i)}_{\theta_t}$. In order to ensure that the inference is decentralized, each agent will receive individualized information about its state and its interactions with the other bodies, this will be done through a GNN. The following is a breakdown of the architecture of our system,

- **GNN**: Each node corresponds to one of the bodies and takes in $[x_t^{(i)}, v_t^{(i)}, m_i]$. The edges encode the distance between the bodies and take in $\left[x_t^{(i)} - x_t^{(j)}, \left\|x_t^{(i)} - x_t^{(j)}\right\|^2\right]$. The network then produces three embedding vectors, h_1, h_2, and h_3, which will be each of the agents' corresponding input.
- **RL Actor**: Each body is modeled by some RL actor, which implements a simple MLP outputting the mean and standard deviation of the distribution learned by the agent. Then one can sample an acceleration vector from this distribution; this will be the agent's action.
- **Centralized Critic**: Which learns a global value of the system based on a centralized reward, this also implements a simple MLP.

During training, we group all three actors in order to create one joint policy,

$$\pi_\theta(a \mid s) = \prod_{i=1}^{3} \mathcal{N}\big(a_i; \mu_{\theta,i}(s), \sigma_{\theta,i}(s)\big),$$

where $\mu_{\theta,i}$ and $\sigma_{\theta,i}$ are the mean and standard deviation output by each actor, respectively, and $\mathcal{N}(a; \mu; \sigma)$ gives the probability of sampling a for a normal distribution with that mean and standard deviation. We can then consider the $\log$ of this probability,

$$\log \pi_\theta(a \mid s) = \sum_{i=1}^{3} \log \mathcal{N}\big(a_i; \mu_{\theta,i}(s), \sigma_{\theta,i}(s)\big).$$

Morover, we consider the ratio of change between the old policy and the current policy as follows,

$$r(\theta) = \exp\big(\log \pi_\theta(a \mid s) - \log \pi_{\theta_{\text{old}}}(a \mid s)\big).$$

We can then use this ratio to compute the policy loss as in PPO, where $\hat{A}$ is the normalized advantage,

$$\mathcal{L}_{\text{policy}}(\theta) = -\mathbb{E}\left[\min\left(r(\theta)\hat{A},\ \text{clip}(r(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}\right)\right]$$

Next, we can consider the loss of our centralized critic with respect to the centralized reward R,

$$\mathcal{L}_{\text{value}}(\phi) = \frac{1}{2}\mathbb{E}_s\big[(V_\phi(s) - R)^2\big]$$

We denote H to be the entropy of our "joint" policy, where $\mathcal{H}(\pi_\theta(\cdot \mid s)$ measures the entropy of the policy's distribution given s,

$$H(\pi_\theta) = \mathbb{E}_s\big[\mathcal{H}(\pi_\theta(\cdot \mid s))\big],$$

Finally, the total loss of our system computed every PPO update is as follows,

$$\mathcal{L} = \mathcal{L}_{\text{policy}} + c_v\mathcal{L}_{\text{value}} - c_e H(\pi_\theta)$$

E. The Reward Function

Our reward function consists of five main sub-rewards, weighed and summed together. This total reward takes values in the range $(-\infty, 0]$ and is of the following form,

$$R = -\left(\omega_o R_o + \omega_f R_f + \omega_s R_s + \omega_p R_p + \omega_{\text{perm}} R_{\text{perm}}\right)$$

where R_o is the target orbit rewards, R_f is the fuel penalization, R_s is the survival reward or collision penalization, R_p is the phase offset penalization and R_{perm} is the permutation rewards.

- $R_o = \frac{1}{3}\sum_{i=1}^{3}\left\|x_i - x^{\text{target}}_{\sigma_t(i)}(k_t)\right\|^2 + \frac{1}{3}\sum_{i=1}^{3}\left(1 - \frac{v_i \cdot v^{\text{target}}_{\sigma_t(i)}(k_t)}{\|v_i\|\left\|v^{\text{target}}_{\sigma_t(i)}(k_t)\right\| + 10^{-12}}\right)$, that is the mean-squared error of the positions of the bodies and those expected in the target orbit, plus the cosine error of the bodies' velocity vectors, and those expected in the target orbit.
- $R_f = \frac{1}{3}\sum_{i=1}^{3}\frac{\|a_i\|^2}{\text{max\_action\_norm}^2 + 10^{-12}}$, that is the mean of the applied accelerations divided by the maximum desired acceleration per action time-step ΔT.
- $R_s = 1$ if there is a collision and 0 otherwise
- $R_p = \frac{\min(|k_t - \text{expected}|, N - |k_t - \text{expected}|)}{N}$, that is the phase difference between the current matched k_t and the expected phase coming from the earlier choice for k_{t-1}.
- $R_{\text{perm}} = \frac{\#\{i \in \{1,2,3\} \mid \sigma_{t-1}(i) \neq \sigma_t(i)\}}{3}$, that is, the proportion of bodies that have switched positions in the ordering from the previous action-step to the current one.

IV. Challenges

The majority of our challenges came from the vast space of solutions that our problem inherently considers. Fully random starts made the task too difficult: most episodes quickly produced collisions, escapes, or states far from the Figure-8.

In light of this, we opted for directed initialization. Start from one fixed configuration near the target choreography, then add small random perturbations to positions and velocities.

This keeps training near the orbit we care about while still preventing the policy from memorizing one exact trajectory. The agents learn local corrective pushes that stabilize the Figure-8 while keeping fuel usage small.

Even, then, reaching stable convergence into the orbit through our PPO MARL approach in reasonable training times, proved to be a challenge. This prompted us to explore more behavioral approaches, such as that described in the latter section on the new setup.

V. Ethics

When tackling this project, we did so with ethics in mind. Due to the purely synthetic nature of our data and data collection, we do not run the risk of breaching user privacy. Due to the celestial nature of our project, it is important to consider the impact of human action in space. Modifying the course and journeys of celestial bodies is of high risk and should be taken with great concern. Our project allows, us to study the consequences of doing so for three celestial bodies and helps us find the most fuel-efficient ways to stabilize them.

VI. The New Setup

The original setup was PPO only and it was trained directly on the environment reward. However, this setup ended up being unstable because the dynamics are very chaotic and work differently with a long-horizon: very small policy changes early on in an episode can lead to a completely different trajectory later. In practice, PPO often improved short horizon behavior without improving final strict convergence, because optimizing local reward at early timesteps didn't address the end goal of converging to an expected figure-8. This made credit assignment noisy and made training brittle.

To address these problems, we decided to look for a new setup. We switched to a teacher guided continuous control pipeline built from MPC/iLQR and TD3. This setup also uses only one actor for simplicity. The control state MPC/iLQR and TD3 [2], [3], [5]. The control state at time t is the concatenated three-body state of

$$s_t = [p_1, p_2, p_3, v_1, v_2, v_3],$$

And the action is the concatenated thrust vector

$$a_t = [u_1, u_2, u_3], \quad u_i \in \mathbb{R}^2,$$

With per-step norm clipping. The target behavior is not just arbitrary trajectory tracking, it is convergence toward a phase-consistent Figure-8 choreography while avoiding the collision/escape and limiting fuel.

The pipeline has three functional stages. In Stage 1 we run MPC/iLQR in the receding horizon mode to generate expert rollouts. At each step, iLQR solves a local finite-horizon control problem with costs for both position and velocity mismatch to the reference orbit, control magnitude, and safety terms for collision [2], [3]. Only the first action of the optimized sequence gets executed, then the problem gets re-solved from the new state with a new set of MPC replanning following the standard MPC receding-horizon control ideas [4]. This produces a demonstration dataset

$$\mathcal{D}_{\text{expert}} = \{(s_t, a_t^\star, s_{t+1})\},$$

Where $a_t^\star$ is the teacher's action. Because this is done in the same simulator used for RL, the demonstrations are dynamics consistent for later training.

In Stage 2 we pretrain the TD3 actor by behavior cloning on $\mathcal{D}_{\text{expert}}$. The actor parameters θ are initialized by minimizing the supervised action error of

$$\mathcal{L}_{\text{BC}}(\theta) = \mathbb{E}_{(s,a^\star)\sim\mathcal{D}_{\text{expert}}} \left[\|\pi_\theta(s) - a^\star\|_2^2 \right].$$

This stage is very important: instead of starting from just random behavior and hoping exploration discovers the stable corrections, the actor starts with a known-good control goal.

In Stage 3 we fine-tune with TD3 [5]. Two critics Q_{ϕ_1}, Q_{ϕ_2} are trained with clipped double-Q targets, target policy smoothing, and delayed actor updates. Using replay buffer samples s (s_t, a_t, r_t, s_{t+1}), TD3 forms targets

$$y_t = r_t + \gamma \min_{i\in\{1,2\}} Q_{\phi_i'}\big(s_{t+1}, \pi_{\theta'}(s_{t+1}) + \epsilon\big),$$

With clipped noise ϵ then it minimizes the critic MSE and updates the actor to maximize $Q_{\phi_1}(s, \pi_\theta(s))$. Target networks are then updated through Polyak averaging. This off-policy phase improves the recovery and robustness past pure imitation, especially for states slightly off of the expert distribution.

At inference, the learned actor directly outputs push actions at each step. The no pushes example runs are implemented by zeroing these actions, which serves as a good ablation to measure against.

The goal of this switch was not just clearer reward, but a model that could fully converge to our desired figure-8 without the instability of PPO, and do so with reasonable fuel usage.

VII. Results and Discussion

All of the reported numbers in Table I were produced with the same fixed initialization distribution setup from the TD3 MPC-clone pipeline configuration: `fixed_init_profile=offset_ref_far`, position jitter standard deviation `0.006`, velocity jitter standard deviation `0.004`, and `64` jitter-attempt retries. The training and evaluation jitter settings were matched (`eval_fixed_init_pos_jitter_std=0.006`, `eval_fixed_init_vel_jitter_std=0.004`),so that each variant was compared under the same perturbed-start regime rather than under a train-test distribution mismatch.

In terms of the training budget, the behavior cloning stage used `150` epochs (batch size `1024`, early-stop patience `25`). TD3 fine-tuning then used warm-started off policy training with replay prefill from the MPC dataset and an evaluation cadence of every `25,000` environment steps (10 eval episodes per checkpoint for training monitoring). For the reported comparison runs the baseline and no fuel configurations used `300,000` total environment steps, while the tuned variant

used 450,000 steps. Final metrics reported in Table I were compute from 300 test-time evaluation rollouts per configuration. Separate qualitative artifact generation used 10 saved inference setups per run for GIF inspection. These jobs were done on NVIDIA A100 GPUs on Rochester Institute of Technology Research Computing infrastructure [6].

Strict convergence success was defined by a couple conditions, not just one threshold crossing. An episode was counted as a strict-success only if: (i) position error ≤ 0.06 and velocity-direction error ≤ 0.09 were achieved for at least 260 consecutive steps, (ii) total episode length was at least 320 steps, (iii) no collision or escape occurred, (iv) the final state was still under the same position/velocity thresholds, and (v) under lock-to-end evaluation, the trajectory did not fall back out of convergence after the streak was first achieved. The reported convergence percentage is the fraction of evaluation episodes that satisfy all those criteria.

TABLE I: Convergence and fuel tradeoffs across evaluated setups.

Setup	Convergence (%)	Avg Fuel Use
No pushes	0.00	—
Pushes (baseline)	35.67	0.06001
Pushes + no fuel penalty	36.33	0.19020
Pushes + ablated hyperparameters	34.33	0.06913

TABLE II: Key hyperparameters by variant (unlisted parameters were shared).

Hyperparameter	Baseline	No-fuel	Ablated
MPC dataset setups	12	12	18
MPC horizon / iters / replan	42 / 14 / 2	42 / 14 / 2	48 / 16 / 2
TD3 total env steps	300k	300k	450k
Replay prefill max samples	150k	150k	225k
Exploration noise (start→end)	0.10→0.02	0.10→0.02	0.06→0.01
Exploration decay steps	300k	300k	250k
$w_{\text{pos}}, w_{\text{vel}}$	20, 12	20, 12	28, 16
w_{fuel}	0.0005	0.0	0.0002
$w_{\text{collision}}$	500	500	700
$w_{\text{switch}}, w_{\text{phase}}$	0.08, 0.20	0.08, 0.20	0.06, 0.25
$w_{\text{near\_collision}}, w_{\text{escape}}$	0, 0	0, 0	0, 0

All three of these variants have the same fixed starting points and jitter settings (offset_ref_far, position jitter 0.006, velocity jitter 0.004, 64 retry budget), the same strict-success thresholds and the same core TD3 architecture / optimizer settings (actor/critic learning rate 3×10^{-4}, hidden size 256, batch size 512).

These results show how important the push-based control is, without pushes the convergence drops to 0%. The no fuel penalty run gives just a small convergence gain over the baseline (36.33% vs. 35.67%) but uses much more fuel (0.19020 vs. 0.06001). THe ablated hyperparameter run does not improve the convergence over baseline and also uses a lot more fuel than the baseline. So among the tested variants, the baseline pushes provide the best efficiency/performance tradeoff. The results are still imperfect, even the best setting converges only about one-third of the time.

VIII. GIF Links (Behavior in Action)

These gifs are intentionally qualitative. For the push enabled variants, we selected runs that did converge so that you can see what successful push-based convergence looks like. For the no-push condition, there were no convergent examples so the GIFs show non-convergence.

Baseline TD3+MPC-clone: https://drive.google.com/drive/u/0/folders/1-SkEGMs4XvkO24bIWm0ITnnUJLLwbLkI
No-fuel-penalty TD3+MPC-clone: https://drive.google.com/drive/u/0/folders/1AZtcJm7CN6CfMQwYOK3Km-ckbin6h2OI
Ablated-hyperparameter TD3+MPC-clone: https://drive.google.com/drive/u/0/folders/1aqgWTYWc7d8r2-Ex187s0S9kAvo6YLOK
No Pushes: https://drive.google.com/drive/u/0/folders/19g9SEy4o_XiIs6LhJpKE1_Fc-xhGhgN4

IX. Reflection

A. Overall Assessment and Evolution/Pivots

Ultimately, we were able to produce stronger convergence results for the figure-8 orbit, even if these came about via methods that greatly differ from those that we originally envisioned. During our first analyses of the problem, it seemed that a purely multi-agent RL approach with the right reward function would be able to quickly learn the correct paths towards the orbit. However, it was through several experiments and training that we realized just how large the solution space for this problem was. We started by enriching the reward function, from a simple one that only considered the "minimal" distance to the orbit positions and velocities with a fuel penalty and a survival reward, to the more sophisticated one described above, that does consider the phase synchronization of the three bodies. Later, we found that learning a policy that would accurately give thrusts for the bodies in any random initial positions would require an unbelievable amount of training and decided to experiment with fixed initial positions, which led to stronger convergence.

Lastly, however, we figured it would be beneficial to experiment with more deterministic teacher approaches with "expert" trajectories. This led to our second approach, where we apply MPC/iLQR in the receding horizon to generate "expert" trajectories to then train our now global TD3 agent with, first via more supervised style learning and then fine-tuning through RL.

B. Future Work

Future approaches to this line of work could tackle the problem of finding a stronger signal for a pure MARL PPO setup to achieve faster convergence on the fixed initial positions case. Another line of work within this problem would be to answer the question of just how much expert knowledge is really needed to achieve convergence within a reasonable training time. Moreover, one could try to come up with a mixed approach between our first and second approaches were the

expert trajectories can be used to inform the reward function or perhaps pre-train the global critic.

C. Key Takeaways

This project allowed us to really experience the importance of reward signal and solution space size and sparsity in Reinforcement Learning approaches. It allowed us to experiment with several iterations of the reward function, each of a stricter nature, and really helped us grasp the exploration rate of these approaches. It also gave us a better grasp of behavioral training approaches and allowed us to explore how to synthetically generate these expert trajectories.

ACKNOWLEDGMENTS

We thank our TA, Daniel Zhu, for their help and guidance throughout the project. We would also like to thank RIT and Brown's CCV for allowing us to train our models on their computing platforms.

REFERENCES

[1] V. Saz Ulibarrena and S. Portegies Zwart, "Reinforcement learning for adaptive time-stepping in the chaotic gravitational three-body problem," *Communications in Nonlinear Science and Numerical Simulation*, vol. 145, p. 108723, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1007570425001340

[2] W. Li and E. Todorov, "Iterative linear quadratic regulator design for nonlinear biological movement systems," in *First International Conference on Informatics in Control, Automation and Robotics*, vol. 2. SciTePress, 2004, pp. 222–229.

[3] Y. Tassa, N. Mansard, and E. Todorov, "Control-limited differential dynamic programming," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2014, pp. 1168–1175.

[4] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. Scokaert, "Constrained model predictive control: Stability and optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.

[5] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*. PMLR, 2018, pp. 1587–1596.

[6] Rochester Institute of Technology, "Research computing services," 2026. [Online]. Available: https://www.rit.edu/researchcomputing/